Step 3 - Architecture Design

GymSpot follows a **feature-based architecture** combined with a lightweight implementation of **Clean Architecture** and the **MVVM pattern**.

The goal is to keep the project scalable and maintainable without introducing unnecessary complexity during the early stages.

▼ Architecture Principles

The architecture is based on three key principles:

- Feature-based organization
- Separation of concerns
- Incremental complexity

Features are grouped by functionality, while core layers separate business logic, data access, and presentation logic.

▼ Technology Stack

Frontend:

- Kotlin
- Compose Multiplatform

Backend services:

- Supabase (authentication and database)
- External APIs for exercises and places

▼ Layer Structure

The project is divided into three main layers:

▼ Presentation Layer

Responsible for UI and user interaction.

Includes:

- Screens
- Components
- Navigation
- ViewModels
- UI state management

▼ Domain Layer

Contains business logic independent from UI and data sources.

Includes:

- Domain models
- Repository interfaces
- Use cases

▼ Data Layer

Handles external integrations and persistence.

Includes:

- Supabase integration
- API clients
- DTO models
- Repository implementations

▼ Feature-Based Organization

Features are separated by domain functionality, such as:

- Authentication
- Exercises
- Routines
- Workouts
- Gyms
- Favorites
- Profiles
- Shared content

Feature-based + Clean Architecture ligera + MVVM

Es decir:

- organizada por funcionalidades
- separando capas
- pero sin volvernos locos con 200 módulos desde el día 1

Porque si lo hacemos demasiado enterprise al principio, te vas a cansar antes de llegar a la parte divertida.

Decisión importante de estructura

Como has dicho “si es una app web”, te doy la recomendación más sensata:

Opción recomendada

Hacerla como **frontend Kotlin + Compose Multiplatform/Web** con Supabase como backend.

Así:

- sigues en tu ecosistema Kotlin
- mantienes Compose

- el proyecto se puede abrir desde navegador
- te sirve para portfolio

Opción más simple si vemos que web complica demasiado

Hacer primero la estructura pensando en **app cliente** y dejar el core listo para que luego tenga:

- target web
- o target Android

O sea, diseñarlo bien desde ya, pero sin bloquearte.

Estructura general del proyecto

Yo te propongo esta:

```
gymspot/
│
├── README.md
├── docs/
│   ├── product-vision.md
│   ├── architecture.md
│   ├── database.md
│   └── api-research.md
│
├── app/
│   ├── src/
│   │   ├── commonMain/
│   │   │   ├── core/
│   │   │   ├── data/
│   │   │   ├── domain/
│   │   │   ├── features/
│   │   │   ├── navigation/
│   │   │   └── ui/
│   │   │
│   │   ├── webMain/
│   │   │   ├── entrypoint/
```

```
│   │   │   └── platform/
│   │   │
│   │   └── androidMain/
│   │       └── platform/
│   │
│   └── build.gradle.kts
│
├── gradle/
├── build.gradle.kts
└── settings.gradle.kts
```

Qué significa cada parte

`docs/`

Aquí va todo lo que luego te da puntos de portfolio:

- visión de producto
- arquitectura
- modelo de datos
- decisiones técnicas

Esto a recruiters y profes les encanta.

`core/`

Todo lo transversal del proyecto.

Ejemplo:

```
core/
├── common/
├── designsystem/
├── network/
├── database/
├── auth/
├── permissions/
├── result/
```

```
├── utils/
└── di/
```

Qué meteríamos aquí

`core/common`

- constantes
- helpers
- extensiones
- clases base

`core/designsystem`

- colores
- tipografías
- botones
- cards
- chips
- loaders
- empty states

`core/network`

- cliente HTTP
- configuración de API
- interceptores si hacen falta

`core/database`

- acceso a Supabase
- config de tablas
- mappers comunes

`core/auth`

- gestión de sesión

- usuario autenticado
- auth manager

`core/permissions`

- localización
- permisos de navegador o plataforma

`core/result`

- `Result`, `UiState`, manejo de errores

`core/di`

- si luego metemos inyección de dependencias

Las capas principales

`domain/`

La capa de negocio pura.

Aquí vive **la lógica importante**, sin depender de UI ni de Supabase directamente.

```
domain/
├── model/
├── repository/
└── usecase/
```

`domain/model`

Tus modelos de negocio:

```
User
Exercise
Routine
RoutineExercise
WorkoutSession
Gym
```

```
Profile
Favorite
```

`domain/repository`

Interfaces, por ejemplo:

```
interfaceExercisesRepository
interfaceRoutinesRepository
interfaceAuthRepository
interfaceGymsRepository
interfaceProfilesRepository
```

`domain/usecase`

Casos de uso:

```
auth/
exercise/
routine/
workout/
gym/
profile/
favorites/
shared_routines/
```

Ejemplos:

- `SignInUseCase`
- `GetExercisesUseCase`
- `CreateRoutineUseCase`
- `CopyPublicRoutineUseCase`
- `GetNearbyGymsUseCase`
- `GetPublicProfileUseCase`

`data/`

La implementación real.

```
data/
├── remote/
├── local/
├── mapper/
└── repository/
```

`data/remote`

Todo lo que venga de fuera:

```
remote/
├── supabase/
│   ├── auth/
│   ├── profiles/
│   ├── routines/
│   ├── workouts/
│   └── favorites/
│
├── exercises_api/
│   ├── dto/
│   ├── service/
│   └── datasource/
│
└── places_api/
    ├── dto/
    ├── service/
    └── datasource/
```

`data/local`

Si luego quieres:

- caché
- preferencias
- configuración local

`data/mapper`

Conversión:

- DTO → Domain
- Domain → Entity
- Entity → UI si hace falta

`data/repository`

Implementaciones de interfaces del dominio:

```
repository/
├── AuthRepositoryImpl
├── ExercisesRepositoryImpl
├── RoutinesRepositoryImpl
├── WorkoutsRepositoryImpl
├── GymsRepositoryImpl
├── ProfilesRepositoryImpl
└── FavoritesRepositoryImpl
```

La parte más importante: `features/`

Aquí es donde realmente vive la app, separada por funcionalidades.

```
features/
├── auth/
├── home/
├── exercises/
├── routines/
├── workouts/
├── gyms/
├── favorites/
├── profile/
└── shared/
```

Y dentro de cada feature:

```
feature_name/
├── presentation/
```

```
├── domain/
└── data/
```

Pero como ya tenemos `domain` y `data` globales, para no duplicar demasiado, yo haría esto:

```
feature_name/
└── presentation/
    ├── screen/
    ├── component/
    ├── state/
    ├── event/
    ├── viewmodel/
    └── navigation/
```

Ejemplo real por feature

`features/routines/`

```
features/routines/
└── presentation/
    ├── screen/
    │   ├── RoutinesScreen.kt
    │   ├── RoutineDetailScreen.kt
    │   ├── CreateRoutineScreen.kt
    │   └── EditRoutineScreen.kt
    │
    ├── component/
    │   ├── RoutineCard.kt
    │   ├── RoutineExerciseItem.kt
    │   └── RoutineVisibilityChip.kt
    │
    ├── state/
    │   ├── RoutinesUiState.kt
    │   ├── RoutineDetailUiState.kt
    │   └── CreateRoutineUiState.kt
    │
```

```
    ├── event/
    │   ├── RoutinesEvent.kt
    │   └── CreateRoutineEvent.kt
    │
    ├── viewmodel/
    │   ├── RoutinesViewModel.kt
    │   ├── RoutineDetailViewModel.kt
    │   └── CreateRoutineViewModel.kt
    │
    └── navigation/
        └── RoutinesNavigation.kt
```

Cómo encaja MVVM aquí

Model

Tu capa de datos + dominio.

View

Las pantallas Compose:

- `LoginScreen`
- `ExercisesScreen`
- `RoutineDetailScreen`

ViewModel

Gestiona:

- estado UI
- eventos
- llamadas a use cases

Patrón de estado que te recomiendo

Yo iría con algo así:

`UiState`

```
dataclassExercisesUiState(
valisLoading:Boolean=false,
valexercises:List<Exercise>=emptyList(),
valsearchQuery:String="",
valselectedMuscleGroup:String?=null,
valerrorMessage:String?=null
)
```

`UiEvent`

```
sealedinterfaceExercisesEvent {
dataclassOnSearchChanged(valquery:String) :ExercisesEvent
dataclassOnMuscleGroupSelected(valmuscle:String?) :Exercise
sEvent
dataclassOnExerciseClicked(valexerciseId:String) :Exercises
Event
dataclassOnFavoriteClicked(valexerciseId:String) :Exercises
Event
}
```

`ViewModel`

- recibe eventos
- llama casos de uso
- actualiza estado

Este patrón te va a dar mucha claridad.

Estructura de navegación

```
navigation/
├── AppDestinations.kt
├── AppNavHost.kt
```

```
├── BottomNavItem.kt
└── Graphs.kt
```

Navegación principal

- Home
- Exercises
- Routines
- Gyms
- Profile

Rutas secundarias

- Exercise detail
- Create routine
- Edit routine
- Workout session
- Public profile
- Public routine detail

Cómo meter la parte social / compartir rutinas

Aquí te lo estructuro ya desde el principio para que no haya que rehacer media app.

Nueva feature: `shared`

```
features/shared/
└── presentation/
    ├── screen/
    │   ├── PublicProfileScreen.kt
    │   ├── SharedRoutineScreen.kt
    │   └── ExploreProfilesScreen.kt
```

```
    ├── component/
    ├── state/
    ├── event/
    ├── viewmodel/
    └── navigation/
```

Casos de uso nuevos para esa parte

En `domain/usecase/shared_routines/`:

```
GetPublicProfileUseCase
GetPublicRoutinesUseCase
GetRoutineByShareCodeUseCase
CopyPublicRoutineUseCase
ToggleRoutineVisibilityUseCase
GenerateShareLinkUseCase
```

Recomendación clave de producto

A nivel estructura, una rutina debería tener algo así:

```
dataclassRoutine(
valid:String,
valownerId:String,
valname:String,
valdescription:String,
valvisibility:RoutineVisibility,
valshareCode:String?,
valcreatedAt:String
)
```

Y:

```
enumclassRoutineVisibility {
PRIVATE,
PUBLIC,
```

```
UNLISTED
}
```

Qué permite esto

- **PRIVATE**: solo tú
- **PUBLIC**: visible en perfil
- **UNLISTED**: visible solo por enlace

Esto está súper bien pensado para más adelante.

Estructura de pantallas recomendada

Auth

- LoginScreen
- RegisterScreen
- ForgotPasswordScreen

Home

- HomeScreen

Exercises

- ExercisesScreen
- ExerciseDetailScreen

Routines

- RoutinesScreen
- RoutineDetailScreen
- CreateRoutineScreen
- EditRoutineScreen

Workouts

- StartWorkoutScreen
- ActiveWorkoutScreen
- WorkoutHistoryScreen
- WorkoutDetailScreen

Gyms

- GymsMapScreen
- GymListScreen
- GymDetailScreen

Favorites

- FavoritesScreen

Profile

- MyProfileScreen
- EditProfileScreen

Shared / Social

- PublicProfileScreen
- SharedRoutineScreen

Estructura UI común

Además de features, haría una carpeta `ui/` para componentes reutilizables:

```
ui/
├── components/
│   ├── AppTopBar.kt
│   ├── SearchBar.kt
│   ├── EmptyState.kt
│   ├── ErrorView.kt
│   ├── LoadingView.kt
│   ├── PrimaryButton.kt
```

```
│   └── ConfirmDialog.kt
│
├── theme/
│   ├── Color.kt
│   ├── Theme.kt
│   ├── Type.kt
│   └── Dimens.kt
│
└── preview/
```

Mi recomendación para no complicarlo demasiado al inicio

En vez de arrancar con una modularización ultra hardcore, yo empezaría con esta versión:

```
commonMain/
├── core/
├── data/
├── domain/
├── features/
├── navigation/
└── ui/
```

Es decir:

- **modular mentalmente**
- pero físicamente en un solo módulo principal al principio

Cuando el proyecto crezca, ahí sí se puede separar.

Eso es mucho más realista.

Cómo quedaría la arquitectura en una frase

Te la dejo casi ya para Notion:

> GymSpot seguirá una arquitectura MVVM con principios de Clean Architecture ligera y organización feature-based. La capa de presentación estará construida con Compose, usando ViewModels, UiState y eventos. La lógica de negocio se concentrará en casos de uso dentro del dominio, mientras que la capa data se encargará de Supabase y APIs externas como ejercicios y gimnasios.

Reparto de responsabilidades

Presentation

- pantallas
- componentes
- navegación
- estado
- eventos
- ViewModel

Domain

- modelos de negocio
- interfaces
- casos de uso

Data

- llamadas a Supabase
- llamadas a APIs
- DTOs
- mappers
- repositorios concretos

Qué ventaja te da esta estructura

Te da exactamente lo que quieres:

- se ve profesional
- es enseñable
- es escalable
- no es un caos
- permite añadir features nuevas
- permite meter la parte social sin rehacer todo

Mi propuesta cerrada de estructura inicial

Yo arrancaría con esto:

```
commonMain/
├── core/
│   ├── auth/
│   ├── network/
│   ├── database/
│   ├── designsystem/
│   ├── utils/
│   └── result/
│
├── data/
│   ├── remote/
│   │   ├── supabase/
│   │   ├── exercises_api/
│   │   └── places_api/
│   ├── mapper/
│   └── repository/
│
├── domain/
│   ├── model/
│   ├── repository/
│   └── usecase/
│
```

```
├── features/
│   ├── auth/
│   ├── home/
│   ├── exercises/
│   ├── routines/
│   ├── workouts/
│   ├── gyms/
│   ├── favorites/
│   ├── profile/
│   └── shared/
│
├── navigation/
└── ui/
```